

LED + Mamba-Hypergraph Narrative Summarizer (v5)

A Research Report on Long-Form Movie Screenplay Summarization

Uday Bindal · May 2026

1. Abstract

We present a dual-tower architecture for automatic narrative summarization of full-length movie screenplays. The system pairs a **Longformer Encoder-Decoder (LED)** for global 16,384-token text encoding with a **dynamic hypergraph tower** that tracks character state evolution over narrative time using **Mamba Selective State Spaces (SSMs)**. Character entities are initialized from LED encoder representations — grounded, not random — and updated through four specialized message streams per scene. The full model contains 516M parameters and is trained on MovieSum (~2,200 movies) with zero-shot transfer capability to MENSA (~500 movies).

After 23 epochs of training the model achieves **ROUGE-1 = 0.187**, **ROUGE-2 = 0.005**, **ROUGE-L = 0.073**, **METEOR = 0.070**, and **Entity F1 = 0.098**. These results reflect the model running with a critical gradient blockage (dead Mamba paths) that was identified but not resolved before training completed; fixing this and retraining Stage 2 is the immediate next step and is expected to significantly improve all metrics.

2. Motivation

2.1 Why Screenplay Summarization is Hard

Movie screenplays are among the most structurally complex documents in NLP. A typical screenplay spans 90–120 pages, contains 50–150 named characters, interleaves action description with dialogue, uses stylized shorthand ("*INT. OFFICE – DAY*"), and encodes causal story logic across temporally separated scenes. Characters introduced in Act 1 return in Act 3; alliances and betrayals span dozens of scenes; emotional arcs are expressed through what characters *do not* say as much as what they do.

Standard summarization approaches fail here for two compounding reasons. First, **length**: a 16,000-token screenplay overwhelms standard transformer context windows (typically 512–1,024 tokens), causing important early scenes to be forgotten by the time late scenes are processed. Second, **narrative structure**: a movie summary is not an extraction of high-attention spans — it

requires understanding *who did what to whom, and why*, tracking entity states across the full temporal arc.

2.2 The Gap in Existing Work

Existing long-document summarizers (BigBird, LED, LongT5) handle the length problem through sparse or global attention, but treat documents as flat token sequences. They have no explicit representation of characters, no mechanism for tracking how a character's state changes from scene to scene, and no way to represent the narrative event graph underlying the story. Prior hypergraph-based NLP work models relational structure but typically operates on short documents with static graphs. No prior work combines long-context LED encoding with a *dynamic, temporally-evolving* hypergraph and Mamba SSM trajectories for entity state tracking at feature scale ($d = 1,024$).

2.3 The Core Research Question

"Can a model that explicitly tracks named character state trajectories across the full narrative arc of a movie screenplay produce better summaries than one that treats the screenplay as a flat token sequence?"

This paper's answer is an architectural bet: yes, and the mechanism is a dynamic hypergraph whose entity nodes evolve through Selective State Space dynamics, conditioned on the global text representations produced by the LED encoder.

3. Architecture

The model consists of three components: a text tower, a hypergraph tower, and a fusion + decoder stage. The total parameter count is **516M** (348.5M trainable in Stage 2, 167.9M frozen LED encoder body).

3.1 Tower 1 — LED Text Encoder

The full screenplay (up to 16,384 tokens) is tokenized as a single sequence with `</s>` separator tokens at scene boundaries. The LED encoder (400M parameters, $d_{\text{model}} = 1,024$, 16 layers) processes this sequence with **global attention** on separator tokens and the BOS token, enabling full cross-scene reasoning. Scene-level representations $\mathbf{H}_{\text{text}} \in \mathbb{R}^{B \times S \times 1024}$ are extracted via mean pooling over each scene's token span — a procedure called **boundary pooling**.

LED was chosen over BigBird and LongT5 because its asymmetric sparse + global attention pattern is specifically designed for hierarchical documents where a few anchor tokens (scene headers) attend to all positions while regular tokens attend locally.

3.2 Tower 2 — Dynamic 4-Stream Hypergraph with Mamba

The hypergraph tower processes $\mathbf{H}_{\text{text}}$ through a dynamic entity graph. Up to 100 named entities per movie are extracted via NER (spaCy), movie-level coreference resolution (fastcoref), and screenplay

artifact filtering (regex removal of INT/EXT headers and revision watermarks). The incidence matrix $I \in \mathbb{R}^{N \times S}$ uses float role weights:

Weight	Narrative Role
1.0	Active speaker — ALL-CAPS character in scene header
0.7	SVO subject of an extracted triplet
0.5	SVO object of a triplet
0.3	Background mention

Entity initialization is LED-grounded: initial entity states are the role-weighted average of scene representations, combined with type embeddings and optionally LED-encoded entity name strings. Entities begin semantically meaningful from epoch 1, not random.

Four message streams are computed per scene per entity and fused via scene-conditioned adaptive gating:

Stream	Mechanism	Purpose
Scene	Entity-aware bilinear attention on hyperedge e_s	What does this entity extract from this scene?
Arc	Temporal attention over past hyperedges + decay bias	How does this entity's arc condition the present?
Social	Co-occurrence weighted mean of co-entity states	Who else is here, and what do they look like?
Relational	Co-occurrence $\times$ cosine alignment of biography h_{accum}	Which co-entities have similar trajectories?

Stream weights are computed by a learned MLP gate ($d \rightarrow d/4 \rightarrow 4$) with a floor=0.15, ceiling=0.55 clamp applied post-softmax to prevent winner-take-all collapse: **$\text{stream_attn} = 0.15 + 0.40 \times \text{softmax}(\text{gate}(e_s))$** . This sums to exactly 1.0 while keeping each stream in the range [0.15, 0.55].

Mamba Temporal Dynamics: per-entity scene-message trajectories $[B \times N, S, D]$ are processed by the EntityMambaBlock (2 layers, $d_{\text{state}}=32$, $d_{\text{inner}}=2,048$). Mamba's input-dependent Δt (state-change magnitude) learns *when* character states change significantly — narrative turning points — vs. when they persist. Emotion modulation (per-entity per-scene sentiment polarity from CardiffNLP RoBERTa) can further bias Δt so emotionally intense scenes produce larger state updates. Δt heatmaps are logged to W&B; for interpretability.

Scene event typing: each scene hyperedge is augmented with an event type embedding (5 types: CONFLICT, ALLIANCE, DECEPTION, DIALOGUE, NEUTRAL) classified from the dominant SVO verb in extracted triplets. **Edge dropout** (rate = 0.10) prevents overfitting to exact co-occurrence patterns.

3.3 Fusion and Decoder

GraphToTextFusion injects hyperedge knowledge back into the text stream: $\text{fused} = \mathbf{H_text} + \sigma(\text{gate_proj}(\mathbf{H_text})) \cdot \text{CrossAttn}(\mathbf{H_text}, \mathbf{H_edges})$. The gate bias is initialized to -3 ($\sigma \approx 0.047 = 4.7\%$ graph weight) — conservative by design, giving the decoder time to stabilize on text-only signal before graph noise can destabilize it.

EntitySceneCrossAttention provides bidirectional cross-attention between scene representations and entity node states. The resulting **aligned memory** [fused_scenes; entity_nodes] (up to 164 positions: 64 scenes + 100 entities) is passed to the **LED decoder** as cross-attention keys/values. The decoder is 100% trainable and generates summaries with beam search (beam = 4, no-repeat 4-gram, max 256 tokens).

4. Data Pipeline

4.1 Extraction

Raw MovieSum and MENSA HuggingFace datasets are converted to compressed JSONL feature files via `emnlp_extractor.py`. Each line is one scene. Key fields per record:

- **clean_text** — raw scene text for LED re-tokenization
- **summary_text** — full un-truncated gold summary (movie-level, stored with scene 0)
- **coref_entities** — mention → canonical map from fastcoref (movie-level resolution)
- **graph_triplets** — Subject_Verb_Object strings from dependency parsing
- **character_emotions** — per-character polarity from CardiffNLP RoBERTa sentiment
- **ner_entities, characters** — spaCy NER output and ALL-CAPS screenplay speaker tags
- **action_mask, dialogue_mask, entity_mask, header_mask** — 4-way modality masks

fastcoref runs **movie-level** coreference (all scenes concatenated), so pronoun chains resolve globally — critical because "he" in scene 40 must link to the character introduced in scene 3, not any male character visible in scene 40.

4.2 Training Split

1,500 movies for training, 298 for evaluation (from MovieSum's ~1,800 unique titles). Scene-level files are byte-offset indexed for $O(1)$ random access across 194,302 scenes without loading everything into memory. Movies exceeding 64 scenes are stride-sampled to fit the `MAX_SCENES = 64` budget.

5. Training Protocol

Training uses a two-stage curriculum that decouples decoder adaptation from entity trajectory learning:

Stage 1 — 3 Epochs (LED encoder frozen)

- Only hypergraph tower, fusion layers, and LED decoder are trained

- Optimizer: AdamW · LR = 1e-4 (new layers), 2e-5 (LED decoder, scene_pool_proj, global attn layers)
- Gradient accumulation: 16 steps (effective batch = 16 movies)
- Loss: cross-entropy (label smoothing 0.05) + entity relational consistency loss (weight 1.5)

Stage 2 — 20 Epochs (LoRA + global attention unfrozen)

- LoRA ($r = 16$, $\alpha = 32$) applied to entity Mamba: in_proj, x_proj, out_proj, dt_proj
 - LED encoder global attention layers unfrozen at LR = 2e-5
 - Cosine warmup schedule with 5% warmup steps
 - Contrastive coherence loss (NT-Xent, scene pairs with Jaccard entity similarity > 0.25) held at $\alpha = 0$ until ROUGE-1 > 0.10
-

6. Training Challenges and Engineering Decisions

6.1 Loss Plateau at 7.5

After Stage 1, training loss plateaued at ~7.5 with no further decrease. Root cause: `gate_proj.bias` was initialized to -4 ($\sigma = 0.018 = 1.8\%$ graph weight), making the hypergraph gradient path 55× weaker than the text path. Fix: bias re-initialized to -3 ($\sigma = 0.047$) via checkpoint surgery (`open_gate.py`), plus optimizer state cleared to avoid stale Adam moments. Loss began decreasing again from epoch 12 onward.

6.2 Arc Stream Winner-Take-All

After the gate fix, W&B; stream weight logs showed the arc stream saturating at 0.85. The previous fusion formula ($0.05 + 0.80 \times \text{softmax}$) allowed any single stream to hit 0.85, causing winner-take-all collapse and starving the other three streams of gradient. Fixed by tightening to $0.15 + 0.40 \times \text{softmax}$ (floor = 0.15, ceiling = 0.55), forcing all four streams to remain active and competitive.

6.3 Dead Mamba Gradients (Critical Issue)

The most severe problem discovered: all entity Mamba parameters showed near-zero gradients for all 23 training epochs. Two blocked gradient paths were diagnosed:

- **Path 1 — Through aligned_memory:** `entity_mem_scale` (a learnable scalar) was initialized to 0. The decoder, having seen zero entity signal for 7+ epochs, learned to ignore entity memory positions entirely. With decoder attention weights on entity slots ≈ 0 , $\partial L / \partial \text{entity_mem_scale} \approx 0$, and therefore $\partial L / \partial \text{Mamba} \approx 0$.
- **Path 2 — Through entity_scene_attn:** `scene_ls` (a learnable d_{model} -dimensional scale vector) was initialized to `zeros(1024)`, making $H_{\text{scenes}} += \text{scene_ls} \times s^* \approx H_{\text{scenes}}$, zeroing the gradient through this second path.

The fix (`fix_mamba_grad.py`): patch `entity_mem_scale`: $0.1 \rightarrow 0.6$ (forcing decoder attention to entity positions at 54% weight, breaking the chicken-and-egg deadlock), and `scene_ls`: `zeros(1024) → full(1024, 0.05)` (opening the second path). The script was committed but failed to reach the

training server before training resumed, so the model ran all 23 epochs without Mamba gradients. **The evaluated model is effectively a LED + static hypergraph baseline, not the full LED + Mamba model.**

6.4 Scheduler State Corruption

Because `fix_mamba_grad.py` never ran (it would have cleared both optimizer and scheduler state), the stale scheduler state was restored from the checkpoint onto a freshly-created Stage 2 schedule, producing an erratic LR pattern: LR fell correctly from $1.5e-5 \rightarrow \sim 0$ at epoch 15, then climbed back to $8.19e-5$ by epoch 23 — 4× higher than intended. Epochs 18–23 trained at dangerously high LR, explaining the metric instability (R1 dips from 0.187 at epoch 21 to 0.145 at epoch 22). The epoch 21 checkpoint is safer.

6.5 ROUGE Evaluation Errors

Initial ROUGE evaluation was computed on only 5 movies (0.7% of eval set) with references truncated to 256 tokens via re-decoding from `target_ids`. Fixed to: 50 sampled movies, full gold `summary_text` from the raw data field (un-truncated). This significantly changed all reported metric numbers.

6.6 scene_pool_proj Gradient Explosion

`scene_pool_proj` (a linear layer initialized to identity for boundary pooling of LED encoder outputs) was grouped with new-layer parameters at $LR = 1e-4$. This caused gradient norms of $\sim 10,000$ and optimizer steps of ~ 1.0 , destroying the identity initialization in the first training batch. Fixed by assigning it to the $LR_DECODER = 2e-5$ group alongside other pretrained components.

7. Results

Best metrics across epochs 12–23 (epoch 21 checkpoint, MovieSum eval set, 298 movies):

Metric	Value
ROUGE-1	0.187
ROUGE-2	0.005
ROUGE-L	0.073
METEOR	0.070
Entity F1	0.098
Train Loss	7.478
Eval Loss	7.609

Metric	Value
Avg entities / scene	9.0
Avg scenes / entity	3.3
Hypergraph coverage	14.8%

ROUGE-2 at 0.005 indicates that while the model produces some correct unigrams, it rarely produces correct bigrams — the output lacks syntactic fluency and coherent multi-word expressions. This is consistent with the model running without Mamba: the decoder generates semantically related but structurally inconsistent text. Entity F1 (0.098) shows a positive upward trend (0.072 → 0.098), indicating the model progressively learns to include correct character names even without Mamba grounding. The LR anomaly (climbing to 8.19e-5 in late epochs) explains the erratic per-epoch ROUGE variance rather than monotone improvement.

8. Areas for Improvement

8.1 Apply Mamba Fix and Retrain Stage 2 (Highest Priority)

Restart Stage 2 from the epoch 11 checkpoint with `fix_mamba_grad.py` properly applied. The 17+ hours of Stage 1 training are preserved. Expected outcome: non-zero Mamba gradients from epoch 12 batch 1, a loss spike to ~8.5 for 1–2 epochs as entity noise is absorbed, then continued decrease and improved ROUGE. The Δt heatmaps should show interpretable narrative turning points at plot climaxes and character revelations.

8.2 Hypergraph Coverage (14.8%)

Only 14.8% of entity-scene incidence slots are non-zero, meaning the graph is very sparse. Improvements: lower the minimum scene frequency threshold for entity inclusion; add scene-level pronoun resolution to recover missed mentions; include implicit entity references from action lines via dependency-parsed subjects of action verbs.

8.3 Re-enable Contrastive Coherence Loss

The NT-Xent narrative coherence loss was never enabled ($\alpha = 0$ throughout). Re-enabling it at ROUGE-1 > 0.10 (now reached) would push scene representations toward a coherence structure where topically related scenes cluster in embedding space, improving summary fluency and entity consistency across generated sentences.

8.4 Full-Corpus ROUGE Evaluation

No full-corpus ROUGE evaluation exists — only the 50-sample approximation logged during training. A dedicated inference pass on all ~700 eval movies with unconstrained beam search (no 256-token truncation) is required for paper-quality numbers. The current inference script also fails because it points to a missing MENSA data path; this needs to be redirected to MovieSum test data.

8.5 Zero-Shot MENSA Transfer

No MENSA evaluation has been run. MovieSum summaries are encyclopedic plot descriptions; MENSA summaries are shorter and more thematic. Zero-shot transfer would reveal whether entity tracking generalizes across narrative styles or overfits MovieSum's summary register.

8.6 LoRA Rank and Scheduling

LoRA was applied with $r = 16$, $\alpha = 32$. Since Mamba never received gradients, all LoRA weights are essentially random at end of training. After the fix, ablate $r = 8$ vs. $r = 32$ and consider including conv1d in LoRA targets. Also validate that `emotion_scale` learns a non-trivial value once Mamba is unblocked — if not, initialize to 0.1 rather than 0 to give emotional intensity an active Δt contribution from the start.

9. Ablation Plan for the Paper

The architecture supports a clean ablation hierarchy across four levels of graph complexity, all switchable via command-line flags:

Run Name	Flag	Research Question
LED-only	<code>--no_hypergraph</code>	Is the hypergraph necessary at all?
LED + static graph	<code>--static_hypergraph</code>	Do dynamic entity updates matter?
LED + GRU (no Mamba)	<code>--no_mamba_entity</code>	Does Mamba outperform GRU for temporal dynamics?
LED + Mamba (full)	(default)	Full model — all components active
No entity names	<code>--no_entity_names</code>	Does name-grounded initialization help?
No edge dropout	<code>--edge_dropout 0.0</code>	Does structural regularization help?
Global stream weights	<code>--no_adaptive_streams</code>	Does per-scene gating beat fixed weights?

The key comparison for the paper is run (3) vs. (4): LED + GRU vs. LED + Mamba. If Mamba's input-dependent gating on entity trajectories produces better ROUGE and Entity F1, that is the core empirical contribution. The Δt heatmap visualizations provide qualitative evidence: large Δt values should appear precisely at scenes corresponding to plot climaxes and character revelations, demonstrating that the SSM has learned to parse narrative structure rather than treating all scenes equally.

10. Conclusion

This project presents a principled approach to long-form narrative summarization that goes beyond treating a screenplay as a flat token stream. The dual-tower architecture — LED for global 16K-token text context and a 4-stream dynamic hypergraph for relational character tracking — addresses known weaknesses of prior summarization systems on long narrative documents. The Mamba SSM component provides an interpretable and computationally efficient mechanism for modeling entity state trajectories, with Δt values directly encoding narrative turning point magnitude.

The primary limitation of the current results is that the Mamba component was gradient-blocked for all 23 training epochs due to a two-path dead-gradient problem in the aligned memory and entity-scene cross-attention modules. As a result, the evaluated model is a LED + static hypergraph system rather than the intended full LED + Mamba architecture. Resolving this one issue — already diagnosed and patched in `fix_mamba_grad.py` — and rerunning Stage 2 is expected to substantially improve performance, particularly on Entity F1 and ROUGE-L, which are most sensitive to coherent entity-grounded generation.

The codebase is fully ablation-ready. The next milestone is a clean Stage 2 training run with working Mamba gradients, followed by full-corpus ROUGE evaluation on MovieSum and zero-shot MENSA transfer, and the GRU vs. Mamba ablation that constitutes the paper's central empirical claim.

System: LED-large-16384 (400M) + DynamicHypergraphTower (116M) · Dataset: MovieSum (1,500 train / 298 eval) ·
Hardware: 1× A100 · Training: 23 epochs, ~100 hours · Best checkpoint: Epoch 21